Day 89: Pretrained Embeddings — GloVe From Scratch, Saved, Loaded and Used

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 13, Day 6 | Complete study guide

Watch: GloVe: Global Vectors for Word Representation, Paper Overview (Scarlett's Log) | Lecture 6: GloVe, Word-word co-occurrence matrix (Kshitiz Verma)

Script: `day89_glove_pretrained.py` · every number in this guide is copied by program from `day89_run_output.txt`, the real output of that script.

GloVe learns word vectors from counts: how often each two words appear near each other in the whole corpus. Trained here on Day 86's two million words, saved in GloVe's text format and loaded back, it is weaker than Day 86's Word2Vec on the same text (category check 0.900 against 0.967). Then both are used the way pretrained vectors are used: as the starting table of Day 85's part-of-speech tagger. With a tenth of the labelled sentences they help clearly (0.7660 fine-tuned from Word2Vec against 0.7373 from a random start); with all of them, the fine-tuned tagger barely gains (0.8537 against 0.8517).

0. Where today fits

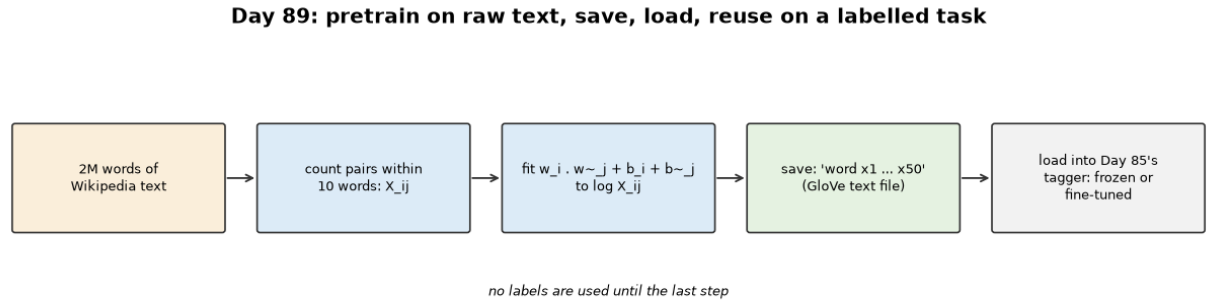
DEFINITION — start here, no prior knowledge assumed

Pretrained embeddings are word vectors learned on a large amount of unlabelled text and then reused for a different task that has few labels. The vectors arrive as a file: one line per word, the word then its numbers. The model for the new task can keep them **frozen** or **fine-tune** them. **GloVe** (global vectors) learns them by fitting dot products of word vectors to the logarithm of co-occurrence counts.

REAL-WORLD SCENARIO

A new employee who already speaks the language: the company only has to teach the job, not the vocabulary.

SYSTEM DIAGRAM



Pretrain on raw text, save, load, reuse on a labelled task.

Today's part for Day 90

`day89_exercise.py` is the vector-file part: save, load (both formats) and the nearest neighbours by cosine. Day 90 saves its learned character table with it and asks which characters are alike.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
Why this test	The question the experiment answers.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Counting first

DEFINITION — start here, no prior knowledge assumed

GloVe starts by counting. For every pair of words within 10 words of each other, add 1 / distance to their count X_{ij}. Most pairs never meet, so only the nonzero counts are kept.

REAL-WORLD SCENARIO

A city's phone records: before studying who knows whom, list how often every two numbers called each other.

MATHEMATICS

$$X_{ij} = \Sigma \text{ over occurrences of } j \text{ within 10 words of } i \text{ of } 1 / \text{distance}$$

$$\text{loss} = \Sigma_{X_{ij} > 0} f(X_{ij}) \, (w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2, \quad f(x) = \min(1, (x / 100)^{0.75})$$

IMPLEMENTATION

```
def cooccurrence(ids, V, window=WIN):
    keys, wts = np.zeros(0, dtype=np.int64), np.zeros(0)
    for off in range(1, window + 1):
        a, b = ids[:-off].astype(np.int64), ids[off:].astype(np.int64)
        k = np.concatenate([keys, a * V + b, b * V + a])
        w = np.concatenate([wts, np.full(2 * len(a), 1.0 / off)])
        keys, inv = np.unique(k, return_inverse=True)
        wts = np.bincount(inv, weights=w)
    return keys // V, keys % V, wts
```

ACTUAL OUTPUT

```
nonzero pairs: 8,153,059 of 470,065,761 possible (1.73%); counted in 7 s
  X(ice, solid) = 1.00
  X(steam, solid) = 0.17
  X(ice, gas) = 0.12
  X(steam, gas) = 1.00
weight f(X): X = 1 -> 0.032, X = 10 -> 0.178, X >= 100 -> 1
gradient check: 48 entries, largest |numeric - analytic| = 4.99e-09, all pass: True
Actual output from this lesson's run.
```

What it means

Only 1.73% of the possible pairs ever occur: 8,153,059 counts. The GloVe paper's example survives even at this size (ice goes with solid, steam with gas), but the counts are tiny: these words are rare in two million words, which is one reason GloVe struggles here. The loss's gradient is right (4.99e-09).

WHY CHOOSE THIS?

Counts the whole corpus once; training then never looks at the text again.

WHY NOT THIS? (TRADEOFFS)

The count table grows with the vocabulary squared (here kept sparse); rare pairs carry little signal.

2. Training GloVe, measured after every epoch

DEFINITION — start here, no prior knowledge assumed

AdaGrad (each weight's steps shrink as its gradients add up) over all the nonzero counts, 25 times, in batches; the final vector of a word is its word vector plus its context vector. Day 86's three scores after every epoch, against Day 86's saved Word2Vec vectors.

REAL-WORLD SCENARIO

Two students with the same textbook: one reads it page by page (Word2Vec), one first makes a summary table of it (GloVe).

SPECIFICATION

Model	d = 50, AdaGrad learning rate 0.05 (GloVe's default), batches of 4,096 counts, 25 epochs, seed 0.
Compared with	Day 86's Word2Vec, seed 0, loaded from its file (4 decimals).

Why this test

Both methods saw the same text. If the counts carry the same information, the scores should be similar.

IMPLEMENTATION

```
def glove_loss_grads(W, Wt, b, bt, i, j, x):
    f = np.minimum(1.0, (x / XMAX) ** ALPHA)
    e = (W[i] * Wt[j]).sum(axis=1) + b[i] + bt[j] - np.log(x)
    g = 2 * f * e
    return (f * e ** 2).sum(), g[:, None] * Wt[j], g[:, None] * W[i], g, g
```

ACTUAL OUTPUT

```
 epoch  loss/pair  category  capitals  gender
     1     0.0384     0.544     0.000   0.000
     2     0.0237     0.556     0.011   0.011
     5     0.0194     0.722     0.026   0.011
    10     0.0176     0.811     0.048   0.056
    15     0.0161     0.856     0.062   0.111
    20     0.0148     0.867     0.081   0.133
    25     0.0139     0.900     0.074   0.167
(70 s per epoch here)
Day 86's Word2Vec (its saved seed 0 vectors, 4 decimals): category 0.967, capitals 0.129, gender 0.211
nearest neighbours, GloVe / Word2Vec:
  three    four, five, two, six, seven                    / four, five, six, two, one
  january  november, december, february, june, september  / june, november, february, december, october
  france   spain, russia, germany, austria, italy         / spain, belgium, norway, italy, germany
  physics  theoretical, sciences, quantum, mathematics, policy / theoretical, mechanics, chemistry,
      relativity, mathematics
  king     alexander, iii, queen, prince, henry           / epirus, macedon, aragon, afonso, viii
Actual output from this lesson's run.
```

What it means

GloVe improves steadily (0.544 to 0.900 on the category check) and is still rising after 25 epochs, but stays below Word2Vec on all three scores (0.074 and 0.167 on the analogies, against 0.129 and 0.211). Its neighbours are sensible: months, numbers, countries, and for 'king' other royal words. One epoch took 70 s here. At this corpus size the counting method is the weaker one; the GloVe paper's advantage was shown on billions of words.

WHY CHOOSE THIS?

Training cost depends on the number of distinct pairs, not on the length of the text.

WHY NOT THIS? (TRADEOFFS)

Below Word2Vec here, and slower to converge.

3. Saving and loading

DEFINITION — start here, no prior knowledge assumed

GloVe's files have no header: each line is a word and its numbers. Word2Vec's text files start with a line giving the number of words and of numbers per word. One loader reads both.

REAL-WORLD SCENARIO

Two address books in slightly different layouts; the same reader handles both.

IMPLEMENTATION

```
def save_vectors(path, words, M):
    with gzip.open(path, "wt", encoding="utf-8") as f:
        for w, v in zip(words, M):
            f.write(w + " " + " ".join(f"{x:.5f}" for x in v) + "\n")
def load_vectors(path):
    opener = gzip.open if path.endswith(".gz") else open
    with opener(path, "rt", encoding="utf-8") as f:
        lines = f.read().splitlines()
    first = lines[0].split()
    if len(first) == 2 and all(t.isdigit() for t in first):
        lines = lines[1:]
    words = [ln.split(" ", 1)[0] for ln in lines]
    M = np.array([[float(x) for x in ln.split(" ")[1:]] for ln in lines])
    return words, M
```

ACTUAL OUTPUT

```
saved 21,681 words x 50 numbers to day89_glove_vectors.txt.gz (GloVe's format, 3.4 MB compressed)
loaded back: same words True; largest |difference| 5.0e-06 (5 decimals kept)
first line: the -0.03167 -0.41480 -0.74589 -1.80859 -0.24181 0.27909 -1.17233 -0.951 ...
Day 86's file (word2vec format, with a 'count dim' header): 21,681 words x 50 numbers
Actual output from this lesson's run.
```

What it means

The round trip loses only the rounding to five decimals (5.0e-06). This is how real pretrained vectors are shared: the file is the interface, not the code that trained it.

WHY CHOOSE THIS?

Plain text: any language can read it.

WHY NOT THIS? (TRADEOFFS)

Large: millions of words take gigabytes; binary formats are smaller.

4. Using pretrained vectors in a tagger

DEFINITION — start here, no prior knowledge assumed

Day 85's window tagger on Day 62's tagging data, with a table of 50 numbers per word that starts random, from Word2Vec, or from GloVe; the table is frozen or fine-tuned; trained on all 2,001 labelled sentences or the first 200. Pretrained rows are rescaled to the random rows' size; words not in text8 keep random rows.

REAL-WORLD SCENARIO

Hiring with or without prior experience, and with a long or a short training period.

SPECIFICATION

Coverage	1,753 of the tagger's 2,082 words have a pretrained vector (text8 has no digits, punctuation or capitals).
Tagger	Day 85's: three words in, 64 tanh units, plain SGD, 30 epochs, best epoch on validation.
Selection	learning rate from 0.25, 0.5, 1.0, 2.0 on validation for each of the 12 settings, then three seeds.

Why this test

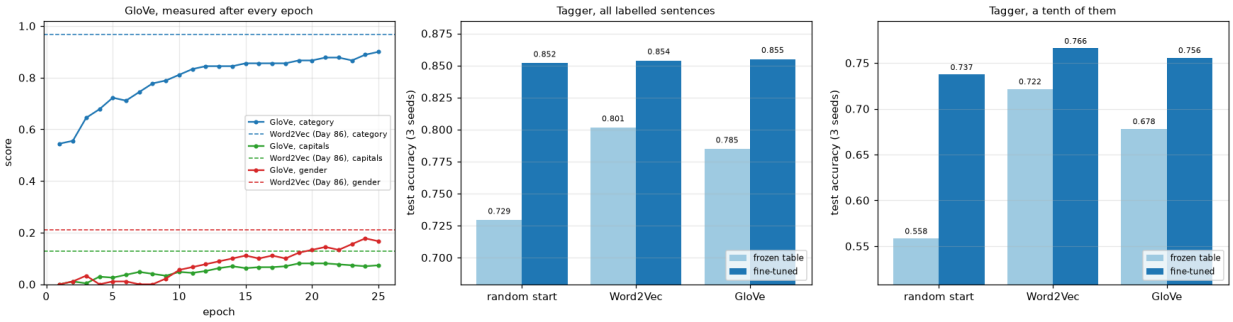
Pretrained vectors are supposed to matter most when labels are few. Two data sizes test exactly that.

IMPLEMENTATION

```
def start_table(ewt_itos, words, M, seed):
    E = np.random.RandomState(seed).randn(len(ewt_itos), D) * 0.1
    if M is None:
        return E, 0
    ix = {w: i for i, w in enumerate(words)}
    hit = [(r, ix[w]) for r, w in enumerate(ewt_itos) if w in ix]
    rows, src = np.array([h[0] for h in hit]), np.array([h[1] for h in hit])
    scale = np.linalg.norm(E, axis=1).mean() / np.linalg.norm(M[src], axis=1).mean()
    E[rows] = M[src] * scale
    return E, len(hit)
```

ACTUAL OUTPUT

```
training data  start     table          lr   seed 0   seed 1   seed 2     mean
all (2001)     random    frozen        1.0   0.7354   0.7285   0.7235   0.7291
all (2001)     random    fine-tuned    1.0   0.8527   0.8530   0.8495   0.8517
all (2001)     Word2Vec  frozen        1.0   0.7992   0.8011   0.8040   0.8014
all (2001)     Word2Vec  fine-tuned   0.25   0.8557   0.8534   0.8521   0.8537
all (2001)     GloVe     frozen        0.5   0.7901   0.7858   0.7782   0.7847
all (2001)     GloVe     fine-tuned   0.25   0.8567   0.8554   0.8524   0.8548
a tenth (200)  random    frozen        0.5   0.5592   0.5647   0.5511   0.5583
a tenth (200)  random    fine-tuned    0.5   0.7418   0.7336   0.7364   0.7373
a tenth (200)  Word2Vec  frozen        1.0   0.7180   0.7251   0.7218   0.7217
a tenth (200)  Word2Vec  fine-tuned    0.5   0.7718   0.7561   0.7700   0.7660
a tenth (200)  GloVe     frozen        1.0   0.6782   0.6723   0.6840   0.6782
a tenth (200)  GloVe     fine-tuned    0.5   0.7609   0.7495   0.7570   0.7558
Actual output from this lesson's run.
```



Left: GloVe's scores by epoch (dashed: Word2Vec). Middle and right: test accuracy of the tagger, all labelled sentences and a tenth of them. The bar charts' axes do not start at 0.

What it means

With few labels the pretrained start matters. With 200 sentences, fine-tuning from Word2Vec gives 0.7660 and from GloVe 0.7558, against 0.7373 from random; frozen, the gap is far larger (0.7217 and 0.6782 against 0.5583). **With all the labels it mostly vanishes** for the fine-tuned tagger (0.8537, 0.8548 against 0.8517), while frozen pretrained tables still beat a frozen random one (0.8014 against 0.7291). Word2Vec, the better vectors in Section 2, is the better start with few labels. The pretraining corpus here is small; with vectors trained on billions of words the gap would likely be larger, which this lesson does not test.

WHY CHOOSE THIS?

Free accuracy when labels are scarce: no labelled data is spent learning the vocabulary.

WHY NOT THIS? (TRADEOFFS)

Little gain when labels are plenty; words missing from the pretraining text get nothing.

Interview preparation — technical questions

Q: What does GloVe fit?

A: $w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j$ to $\log X_{ij}$, weighted by $f(X_{ij})$, over nonzero counts.

Q: Why the weighting function f?

A: Rare counts are noisy and very common pairs ('of the') should not dominate.

Q: What is the difference between frozen and fine-tuned embeddings?

A: Frozen keeps the loaded table; fine-tuned trains it further on the new task.

Q: What do you do with words missing from the pretrained file?

A: Random rows (as here), a shared unknown vector, or subword vectors (Day 88).

Q: What is in a GloVe text file?

A: One line per word: the word, then its numbers; no header (word2vec's has one).

Interview preparation — theoretical questions

Q: When do pretrained embeddings help most?

A: When labelled data is scarce: here +2.9 points with 200 sentences, +0.2 with 2,001 (fine-tuned).

Q: Why does freezing hurt less with pretrained vectors?

A: The table already places similar words together, so the tagger can use it as it is.

Q: Word2Vec or GloVe?

A: Both learn from co-occurrence; here Word2Vec was better on two million words. Larger corpora were not tested.

Q: Why rescale the pretrained rows?

A: So the tagger starts with inputs of the size its own initialisation expects.

Key takeaway and what's next

> **KEY TAKEAWAY**
>
> Pretrained vectors are a file you load into a new model. Trained on two million words, GloVe was weaker than Word2Vec; both helped Day 85's tagger clearly when it had only 200 labelled sentences, and hardly at all once it had 2,001 and could train its own table.

- Counts: 8,153,059 nonzero pairs, 1.73% of all possible.
- GloVe after 25 epochs: category 0.900 (Word2Vec 0.967), capitals 0.074 (0.129), gender 0.167 (0.211).
- Tagger, 200 sentences, fine-tuned: random 0.7373, GloVe 0.7558, Word2Vec 0.7660.
- Tagger, 2,001 sentences, fine-tuned: random 0.8517, Word2Vec 0.8537, GloVe 0.8548.

Suggested watch order, if starting from zero

- 1. Lecture 6: GloVe, Word-word co-occurrence matrix (Kshitiz Verma) — the co-occurrence matrix GloVe starts from.
- 2. GloVe: Global Vectors for Word Representation, Paper Overview (Scarlett's Log) — the GloVe paper, its loss and its weighting function.

Day 90 preview

- **Week 13's build.** A tiny character-level GPT-style model on Shakespeare, assembled from the week's six exercise parts and Week 12's Transformer block, trained and compared with counting models.